

★ WEEK 8 · PART 2 of 3

Filtering, Smoothing, Forecasting

Three Questions, One Recursive Engine

Real-time estimates · hindsight · projection · a drifting hedge ratio

Given the two equations, three questions follow: what is the state now, what was it then, and what will it be? Each conditions on a different slice of data — and one recursive cycle answers them all.

2

PART 2 · INFERENCE

Estimating the Hidden State

Filtering, smoothing, and forecasting, the predict-update cycle that powers them, and the time-varying hedge ratio that carries the pairs thread into state space.

We Have the Model — Now, the Inference

Part 1 set up the state; Part 2 estimates it

1

The state is hidden

Part 1 gave us the two equations: a state evolving over time, seen only through noisy observations. But we never wrote down HOW to recover the state.

2

Three questions, one framework

What is the state now (filtering)? What was it, in hindsight (smoothing)? What will it be (forecasting)? All three fall out of the same model.

3

One recursive cycle answers them

A predict-then-update loop estimates the state in real time — the engine we will make optimal as the Kalman filter (Week 9).

Part 2 in one line: filter, smooth, and forecast the hidden state — all from one recursive cycle.

Three Questions, One Framework

Same goal, different information sets

Estimating a hidden state comes in three flavors, distinguished only by WHICH data you condition on: everything up to now, the whole sample, or nothing beyond now.

The three tasks

- Filtering: the state NOW, from data up to now.
- Smoothing: the state THEN, using all data in hindsight.
- Forecasting: the state AHEAD, projecting beyond now.

The unifying point

- The model is identical in all three.
- Only the conditioning information changes.
- So one engine, run differently, does all three.

The Three Inference Problems

One line each

$$\underbrace{\mathbb{E}[x_t \mid y_{1:t}]}_{\text{filtering}} \quad \underbrace{\mathbb{E}[x_t \mid y_{1:T}]}_{\text{smoothing}} \quad \underbrace{\mathbb{E}[x_{T+h} \mid y_{1:T}]}_{\text{forecasting}}$$

Reading them

- Filtering conditions on data up to time t .
- Smoothing conditions on the entire sample, up to T .
- Forecasting conditions on data up to T , then projects forward.

Why the distinction matters

- Filtering is what you can do LIVE, as data arrives.
- Smoothing is the best estimate, but only after the fact.
- Forecasting is filtering's natural extension into the future.

Filtering: The State Now

The best real-time estimate

What it is

- The estimate of the state at time t , using data up to t .
- It updates the instant a new observation arrives.
- It is causal — it never peeks at the future.

Why it is central

- It is what you use in live trading and control.
- It powers real-time signals and risk estimates.
- It is the output of the recursive filter.

Smoothing: The State Then

The best estimate, with hindsight

What it is

- The estimate of the state at a past time, using ALL the data.
- It borrows information from AFTER that moment.
- So it is smoother and more accurate than filtering.

When to use it

- For analysis and decomposition after the fact.
- To read a clean trend or seasonal component from history.
- Never for real-time decisions — it uses future data.

Which Data Does Each Use?

Filtering conditions on data up to now; smoothing uses the whole sample, past and future; forecasting projects beyond now. Same model, three information sets.

Which Data Does Each Use?

Filtering, smoothing, and forecasting differ only in the information set they condition on.

past

now (t)

future

Filtering
state NOW

uses data up to now (y up to t) - real-time

Smoothing
state THEN

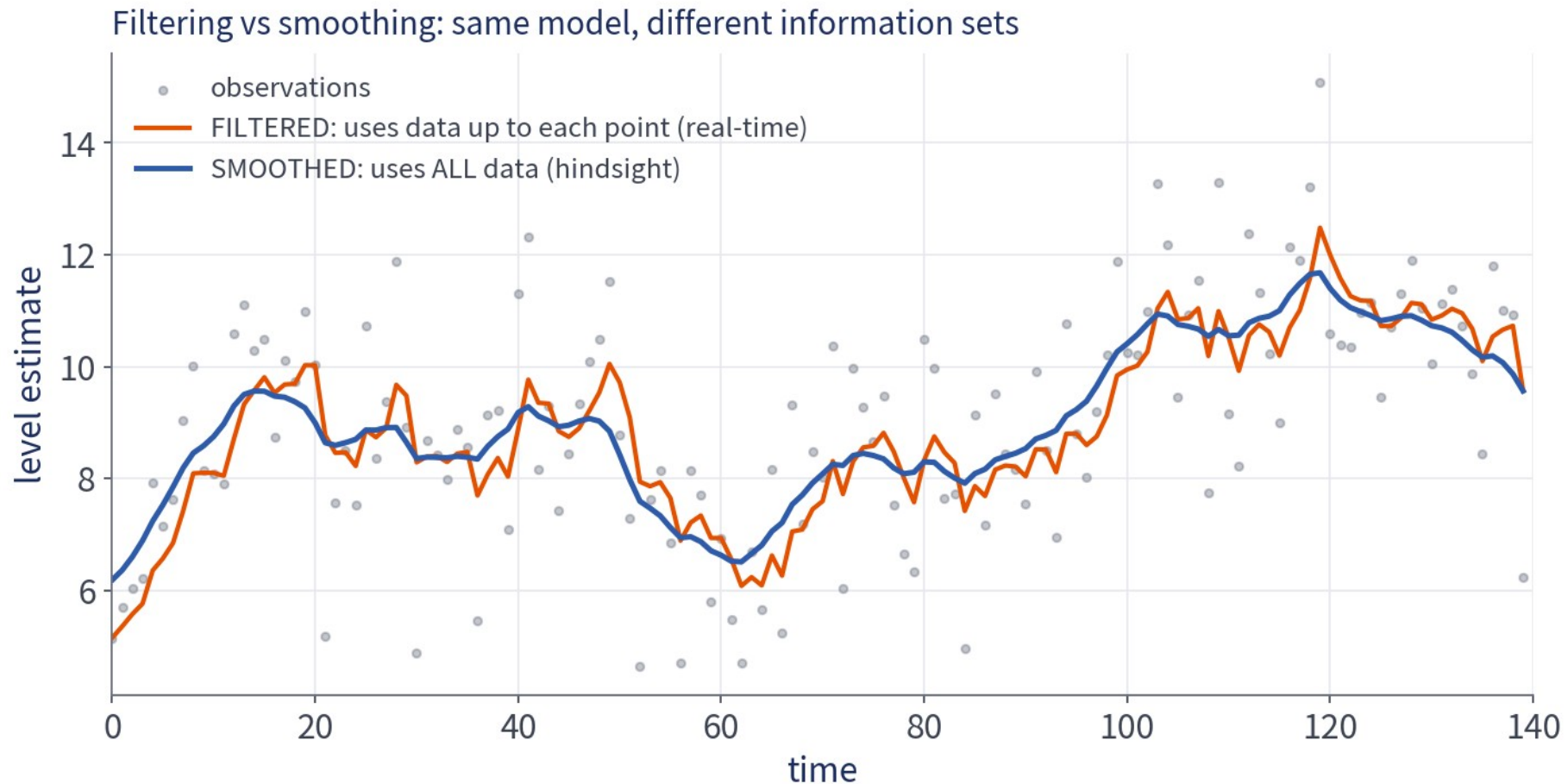
uses ALL data, past and future (y over the whole sample) - best hindsight estimate

Forecasting
state AHEAD

projects beyond now (y up to T)

Filtered vs Smoothed

The filtered estimate (orange) is real-time — it lags and wobbles at the edges. The smoothed estimate (blue) uses all data, so it is cleaner — but only available after the fact.



Filtering vs Smoothing

The same estimate, two information sets

**Filtering answers 'what is the state now, given what I know now?'.
Smoothing answers 'what was the state, given everything I
eventually learned?'. Hindsight makes smoothing cleaner.**

Filtering

- Causal, real-time, usable live.
- Noisier and slower to react at the edges.
- The right tool for trading and monitoring.

Smoothing

- Uses future data — only available in hindsight.
- Cleaner and more accurate for every past point.
- The right tool for analysis and reporting.

Real-Time Updating

Revise the estimate as each point arrives

The recursion

- Carry forward the current best estimate and its uncertainty.
- When new data arrives, correct the estimate.
- No need to refit on the whole history each time.

Why it is efficient

- Each step is a light, constant-cost update.
- It scales to streaming, high-frequency data.
- This recursive structure is the filter's great virtue.

The Predict-Update Cycle

One step of the recursion

$$\text{predict: } x_{t|t-1} = T x_{t-1|t-1} \quad \text{update: } x_{t|t} = x_{t|t-1} + K_t (y_t - Z x_{t|t-1})$$

Predict

- Push the state forward by the transition.
- Its uncertainty grows — you know a little less.
- This gives a forecast of the next observation.

Update

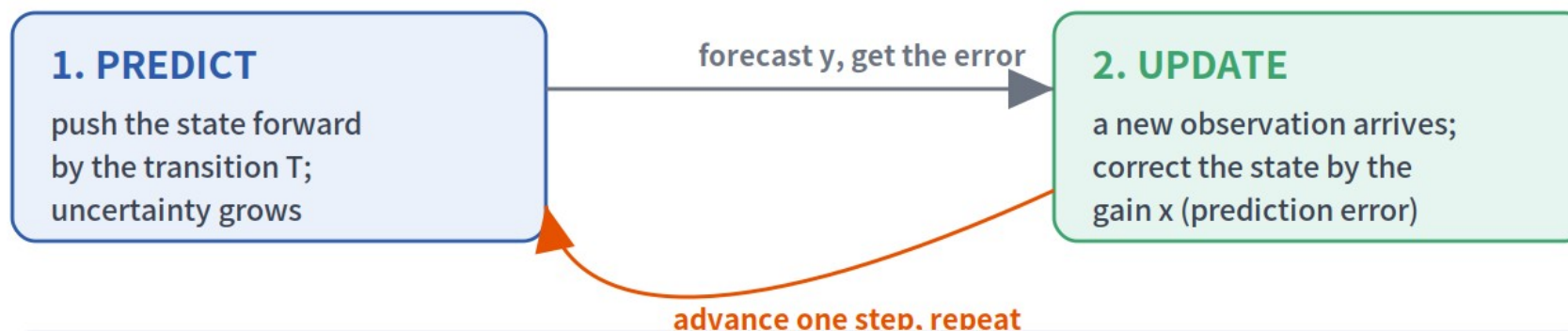
- Compare the actual observation to that forecast.
- Correct the state in proportion to the error, via the gain K .
- The gain decides how much the new point moves you.

The Predict-Update Cycle

Guess ahead, see the new data, correct by the gain — then advance and repeat. The gain K balances trust in the model against trust in the data; the Kalman filter makes it optimal.

The Predict-Update Cycle

Filtering is a loop: guess ahead, see the new data, correct - then repeat.



The gain K is the whole trick

- large gain: the state is uncertain or the data is precise → trust the new point
 - small gain: the state is well-pinned or the data is noisy → barely move
- This optimal gain is exactly what the Kalman filter computes (Week 9).

Forecasting From a State Space Model

Filtering, continued into the future

To forecast, just keep predicting without updating — there is no new data to correct with. The state coasts forward by its dynamics, and the uncertainty grows each step.

How it works

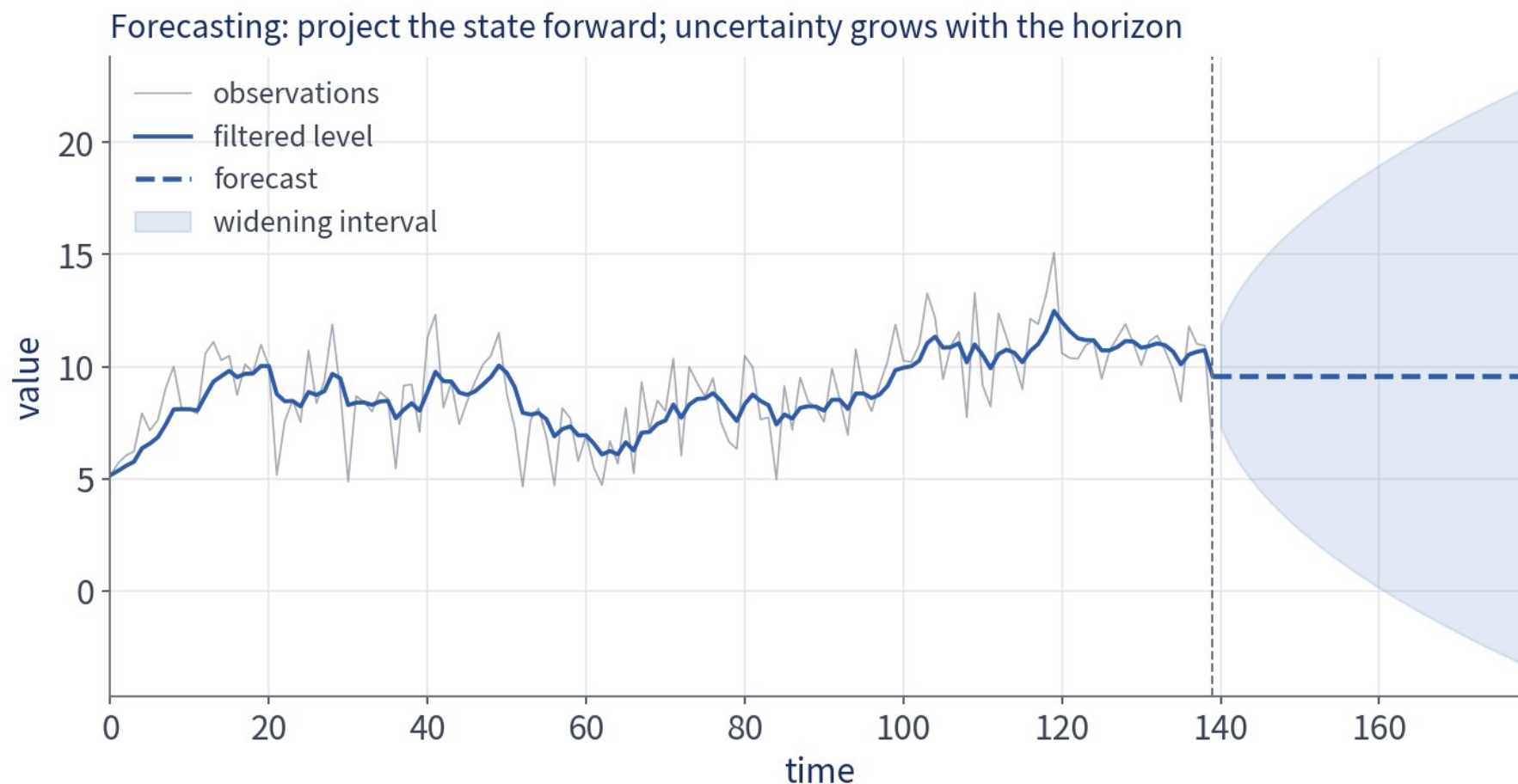
- Run the predict step repeatedly, with no updates.
- The state evolves by the transition alone.
- Map it through the observation equation to forecast the data.

What you get

- A forecast path for both the state and the observations.
- Intervals that widen naturally with the horizon.
- Consistent with the same model that filtered the past.

Forecasting With Growing Uncertainty

Past the last observation, the filter keeps predicting with no data to correct it. The point forecast coasts on the state's dynamics; the interval fans out with the horizon.



Uncertainty, for Free

Error bars come with every estimate

you also get the uncertainty for free: $P_{t|t} = \text{Var}(x_t \mid y_{1:t})$

What you get

- Alongside each state estimate, its variance.
- That gives calibrated error bars at every point.
- The intervals on the forecasts come from it directly.

Why it matters

- You always know how much to trust the estimate.
- The gain uses this uncertainty to weight new data.
- Risk and sizing decisions can use it explicitly.

Time-Varying Parameters, Applied

Let a coefficient be a hidden state

The most valuable trick: make a model's coefficient itself a hidden state that drifts over time. The filter then estimates that coefficient at every moment — a model that adapts.

The setup

- Write the coefficient as a state following a random walk.
- The observation equation uses that drifting coefficient.
- Filtering tracks its value through time.

The finance payoff

- Betas, exposures, and hedge ratios all change with regime.
- A single fixed number is stale the moment things shift.
- State space gives a live, adapting estimate.

The Dynamic Hedge Ratio

The pairs thread enters state space

$$\text{dynamic hedge ratio: } Y_t = \beta_t X_t + \varepsilon_t, \quad \beta_t = \beta_{t-1} + \eta_t$$

The model

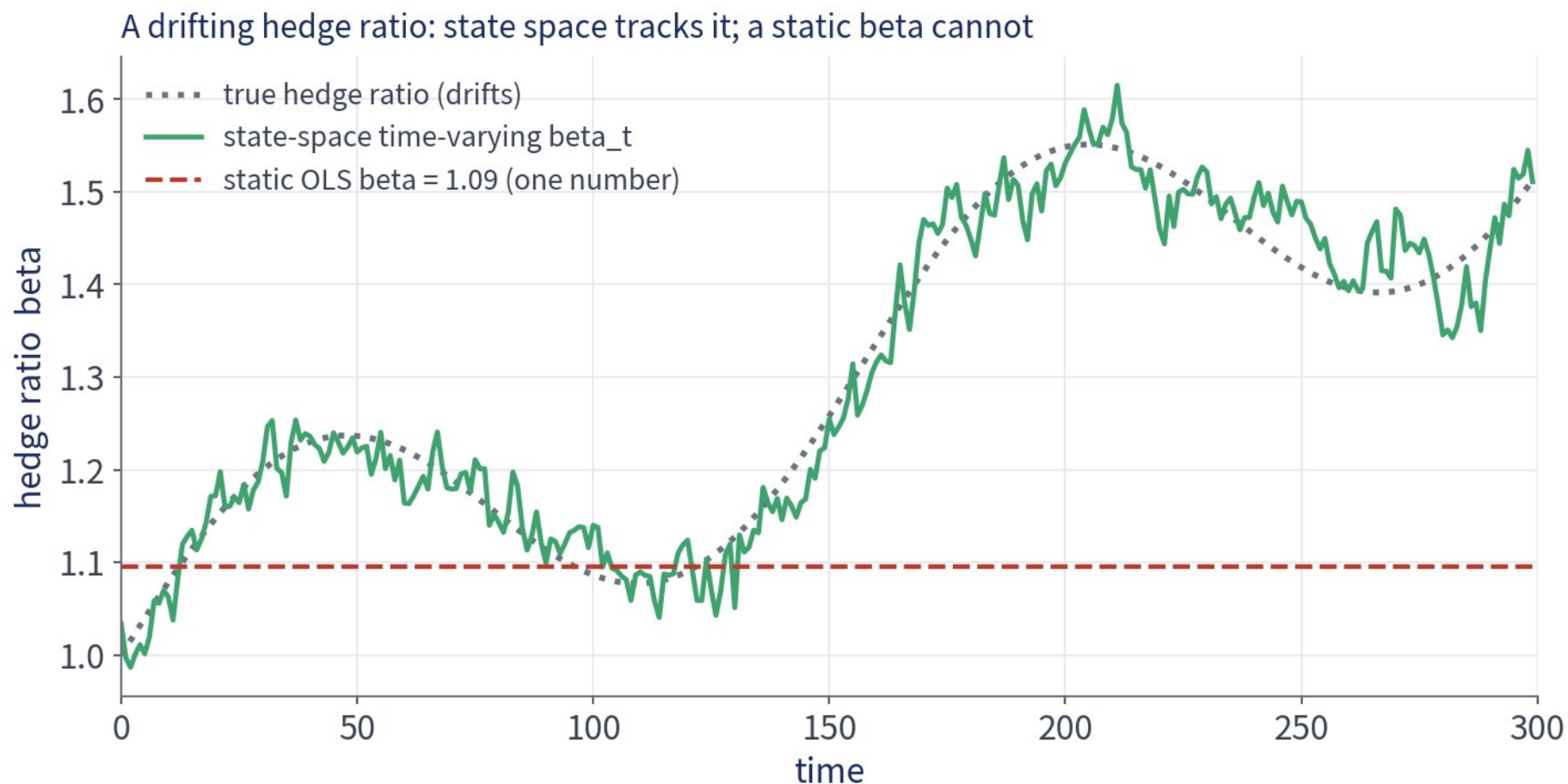
- The hedge ratio beta becomes a drifting hidden state.
- It follows a random walk instead of a fixed value.
- The filter re-estimates it as new prices arrive.

Why it beats a fixed beta

- The true relationship between two assets shifts over time.
- A static hedge ratio (Week 7) slowly goes stale.
- A drifting beta keeps the spread properly hedged.

A Drifting Hedge Ratio

When the true hedge ratio drifts, a single static OLS beta is a poor compromise. The state-space estimate tracks the drift in real time — a self-updating hedge.



Why a Drifting Beta Wins

Adapting beats assuming

The static problem

- One OLS beta averages over the whole history.
- It is wrong at the start and wrong at the end of a drift.
- The 'hedge' leaks exposure whenever the ratio moves.

The dynamic fix

- Track beta as a state; it follows the relationship live.
- The spread stays genuinely market-neutral over time.
- Signals built on it are cleaner and more reliable.

The Pairs-Trading Thread, So Far

Three stages, three weeks

Stage	Week	What it added
Stage I	5	GARCH bands — volatility-scaled thresholds on the spread
Stage II	7	Cointegration — a validated, mean-reverting equilibrium spread
Stage III	8	State space — a drifting, self-updating hedge ratio

Each week deepened the same strategy: better bands, a real anchor, and now an adaptive hedge.

Diagnostics for State Space Models

Checking the fit through the innovations

The key object

- The innovations are the one-step prediction errors.
- In a well-fit model they are standardized white noise.
- They are the state space analogue of residuals.

What to check

- No autocorrelation in the standardized innovations.
- Roughly normal, with constant variance.
- Leftover structure means the state design is incomplete.

Filter, Smooth, and a Dynamic Hedge

Direct Claude Code across all three tasks

GOAL Compare filtering vs smoothing on one series, then estimate a time-varying hedge ratio for a pair.

[INPUT] Prompt · Claude Code

Write and run `filter_smooth_lab.py` that:

1. fits a local level model and plots the FILTERED and SMOOTHED states together, so the difference is visible;
2. produces a multi-step forecast with widening confidence bands;
3. for a pair, fits a time-varying regression (random-walk beta) and plots `beta_t` against the static OLS beta;
4. checks the standardized innovations for autocorrelation.

Say when the dynamic beta diverges from OLS.

How to work with it

- Smoothed should be cleaner than filtered.
- Forecast bands should fan out with the horizon.
- The dynamic beta should drift where OLS is flat.
- Clean innovations = a well-specified model.

Deliverable: `filter_smooth_lab.py` + filter/smooth, a forecast, and a drifting hedge ratio.

Three Questions Before You Move On

If you can answer these, Part 2 has landed

Q1

You need a state estimate for live trading. Filtering or smoothing?

Filtering (causal)

Q2

What does the gain K balance in the update step?

Model vs data trust

Q3

Why model a hedge ratio as a drifting state instead of one OLS number?

The relationship changes

All three come from the information sets, the predict-update cycle, and time-varying parameters.

Three Misconceptions to Avoid

Where inference intuition slips

Common myths

- "Smoothing is just a better filter — use it live."
- "Forecasting needs a different model from filtering."
- "A time-varying beta is over-fitting."

The reality

- Smoothing uses FUTURE data — never available in real time.
- Forecasting is the same filter, predicting with no updates.
- A drifting beta with tuned noise adapts without chasing noise.

Three Things to Carry Forward

Part 2 in one screen

1

Filter, smooth, forecast — one model

They differ only in the data they condition on: up to now (filter), the whole sample (smooth), or projected ahead (forecast). One engine serves all three.

2

A predict-update cycle does it, with uncertainty

Guess ahead, then correct by the gain when data arrives. Each estimate comes with calibrated error bars — the gain uses that uncertainty to weight new data.

3

Time-varying parameters carry the pairs thread

Make the hedge ratio a drifting state, and the filter tracks it live. That is Stage III of pairs trading — an adaptive, self-updating hedge.

Next — Part 3: the midterm synthesis, tying Weeks 1-7 into one connected story.